```
//runways

function runways_required(arrival_time, departure_time, no_of_planes) {
    let events = [];
    for (let i = 0; i < no_of_planes; i++) {
        events.push({ time: arrival_time[i], type: 'arrive' });
        events.push({ time: departure_time[i], type: 'depart' });
    }
    // Sort events: first by time, if times are equal, 'depart' should come before 'arrive'
    events.sort((a, b) => {
        if (a.time === b.time) {
            return a.type === 'depart' ? -1 : 1;
        }
        return a.time - b.time;
    });
    let currentRunways = 0;
    let maxRunways = 0;
    for (let event of events) {
        if (event.type === 'arrive') {
            currentRunways++;
        } else {
            currentRunways--;
        }
        maxRunways = Math.max(maxRunways, currentRunways);
    }
    return maxRunways;
}
//pages
function minimum_pages(arr, k) {
    let low = Math.max(...arr); // The minimum possible max pages is the highest single book page
count
    let high = arr.reduce((sum, pages) => sum + pages, 0); // The maximum possible is the sum of all
pages
    function canAllocate(maxPages) {
        let currentSum = 0;
        let requiredStudents = 1;

        for (let pages of arr) {
            if (currentSum + pages > maxPages) {
                // Need a new student
                requiredStudents++;
                currentSum = pages;
                if (requiredStudents > k) return false;
            } else {
                currentSum += pages;
            }
        }
        return true;
    }

    while (low < high) {
```

```
        let mid = Math.floor((low + high) / 2);
        if (canAllocate(mid)) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }

    return low;
}
//strings
function isPermutation(s1, s2) {
    if (s1.length > s2.length) return false;
    const charCountS1 = new Array(26).fill(0);
    const charCountWindow = new Array(26).fill(0);
    const aCharCode = 'a'.charCodeAt(0);
    // Initialize frequency counts for s1
    for (let i = 0; i < s1.length; i++) {
        charCountS1[s1.charCodeAt(i) - aCharCode]++;
    }
    // Initialize the frequency counts for the first window in s2
    for (let i = 0; i < s1.length; i++) {
        charCountWindow[s2.charCodeAt(i) - aCharCode]++;
    }
    // Initial comparison of the first window
    if (arraysEqual(charCountS1, charCountWindow)) {
        return true;
    }
    // Sliding the window over s2 to check other substrings
    for (let i = s1.length; i < s2.length; i++) {
        // Increment count of new char in window
        charCountWindow[s2.charCodeAt(i) - aCharCode]++;
        // Decrement count of the old char leaving the window
        charCountWindow[s2.charCodeAt(i - s1.length) - aCharCode]--;
        // Compare the updated window with s1's frequency count
        if (arraysEqual(charCountS1, charCountWindow)) {
            return true;
        }
    }
    return false;
}
function arraysEqual(arr1, arr2) {
    for (let i = 0; i < 26; i++) {
        if (arr1[i] !== arr2[i]) {
            return false;
        }
    }
    return true;
}
```

```
//majority element
int majorityElement(const vector<int>& arr) {
    int n = arr.size();
    int candidate = -1;
    int count = 0;
    // Find a candidate
    for (int num : arr) {
        if (count == 0) {
            candidate = num;
            count = 1;
        } else if (num == candidate) {
            count++;
        } else {
            count--;
        }
    }
    // Validate the candidate
    count = 0;
    for (int num : arr) {
        if (num == candidate) {
            count++;
        }
    }
    // If count is greater than n / 2, return the candidate; otherwise, return -1
    if (count > n / 2) {
        return candidate;
    } else {
        return -1;
    }
}
```

```
//Combining Customer Orders and Sales

SELECT CustomerID, SUM(Amount) AS TotalSpending
FROM (
    SELECT CustomerID, Amount FROM Orders
    UNION ALL
    SELECT CustomerID, Amount FROM Sales
) AS Combined
GROUP BY CustomerID;


// List Employees, Departments, and Managers

SELECT e.emp_name, d.department_name, COALESCE(m.emp_name, 'No Manager') AS
manager_name
FROM employees e
JOIN departments d ON e.department_id = d.dept_id
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

//Top 5 Departments by Average Salary

```
SELECT d.department_name, AVG(s.salary) AS average_salary
FROM salaries s
JOIN employees e ON s.emp_id = e.emp_id
JOIN departments d ON e.department_id = d.dept_id
GROUP BY d.department_name
ORDER BY average_salary DESC
LIMIT 5;
```

//Count Leads in Specific Months

```
SELECT COUNT(*) AS lead_count
FROM Leads
WHERE MONTH(Created_Date) IN ('April', 'May', 'June');
```

//Identify Even Employee IDs

```
SELECT EmpID
FROM EMPLOYEEINFO
WHERE EmpID % 2 = 0;
```